

MODULE-5

8 Hours

File System, Implementation of File System: File system: File concept; Access methods; Directory and Disk structure; File system mounting; File sharing; **Implementing File system:** File system structure; File system implementation; Directory implementation; Allocation methods; Free space management.

Secondary Storage Structure, Protection: Mass storage structures; Disk structure; Disk attachment; Disk scheduling; Disk management; **Protection:** Goals of protection, Principles of protection, Domain of protection, Access matrix.

Textbook 1: Chapter – 10 (10.1-10.5) ,11 (11.1-11.5),12 (12.1-12.5), 14 (14.1-14.4)

FILE SYSTEM

5.1 FILE SYSTEM, File Concept

- Computers can store information on various storage media, such as magnetic disks, magnetic tapes, and optical disks.
- Operating system provides a uniform logical view of information storage. The operating system abstracts from the physical properties of its storage devices to define a logical storage unit, the file.
- Many different types of information may be stored in a file - source programs, executable programs, numeric data, text, graphic images, sound recordings, and so on.

5.1.1 File Attributes

A file's attributes vary from one operating system to another but typically consist of these:

- **Name**. The symbolic file name is the only information kept in human readable form.
- **Identifier**. This unique tag, usually a number, identifies the file within the file system; it is the non-human-readable name for the file.
- **Type**. This information is needed for systems that support different types of files.
- **Location**. This information is a pointer to a device and to the location of the file on that device.
- **Size**. The current size of the file (in bytes, words, or blocks) and possibly the maximum allowed size are included in this attribute.
- **Protection**. Access-control information determines who can do reading, writing, executing, and so on.
- **Time, date, and user identification**. This information may be kept for creation, last modification, and last use. These data can be useful for protection, security, and usage monitoring.

5.1.2 File Operations

To define a file properly, we need to consider the operations that can be performed on files. The operating system can provide system calls to create, write, read, reposition, delete, and truncate files.

1. **Creating a file**. Two steps are necessary to create a file. First, space in the file system must be found for the file. Second, an entry for the new file must be made in the directory.

2. **Writing a file.** To write a file, we make a system call specifying both the name of the file and the information to be written to the file. Given the name of the file, the system searches the directory to find the file's location. The system must keep a write pointer to the location in the file where the next write is to take place.
3. **Reading a file.** To read from a file, we use a system call that specifies the name of the file and location where (in memory) the next block of the file should be put. Again, the directory is searched for the associated entry, and the system needs to keep a read pointer to the location in the file where the next read is to take place. Because a process is usually either reading from or writing to a file, the current operation location can be kept as a per-process current-file-position-pointer.
4. **Repositioning within a file.** The directory is searched for the appropriate entry, and the current-file-position pointer is repositioned to a given value. This file operation is also known as a file seek.
5. **Deleting a file.** To delete a file, we search the directory for the named file. Having found the associated directory entry, we release all file space, so that it can be reused by other files, and erase the directory entry.
6. **Truncating a file.** The user may want to erase the contents of a file but keep its attributes. Rather than forcing the user to delete the file and then recreate it, this function allows all attributes to remain unchanged -except for file length-but lets the file be reset to length zero and its file space released.
7. Most of the file operations mentioned involve **searching** the directory for the entry associated with the named file. To avoid this constant searching, many systems require that an `open()` system call be made before a file is first used actively.
8. The **`open()`** operation takes a file name and searches the directory, copying the directory entry into the open-file table. When a file operation is requested, the file is specified via an index into this table, so no searching is required.
9. The operating system uses two levels of internal tables: a per-process table and a system-wide table. The per- process table tracks all files that a process has opened. Stored in this table is information regarding the use of the file by the process.
10. Each entry in the per-process table in turn points to a system-wide open-file table. The system-wide table contains process-independent information. Once a file has been opened by one process, the system-wide table includes an entry for the file.

Several pieces of information are associated with an open file.

- **File pointer.** On systems that do not include a file offset as part of the `read()` and `write()` system calls, the system must track the last read-write location as a current-file-position pointer. This pointer is unique to each process operating on the file.
- **File-open count.** As files are closed, the operating system must reuse its open-file table entries, or it could run out of space in the table. Because multiple processes may have opened a file, the system must wait for the last file to close before removing the open-file table entry.
- **Disk location of the file.** Most file operations require the system to modify data within the file. The information needed to locate the file on disk is kept in memory so that the system does not have to read it from disk for each operation.
- **Access rights.** Each process opens a file in an access mode. This information is stored on the per-process table so the operating system can allow or deny subsequent I/O requests.

Open File Locking

- Provided by some operating systems and file systems
- Mediates access to a file
- Mandatory or advisory:
 - **Mandatory** – access is denied depending on locks held and requested
 - **Advisory** – processes can find status of locks and decide what to do

File Types

- A common technique for implementing file types is to include the type as part of the file name. The name is split into two parts—a name and an extension, usually separated by a period character.
- In this way, the user and the operating system can tell from the name alone what the type of a file is. The system uses the extension to indicate the type of the file and the type of operations that can be done on that file.

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, perl, asm	source code in various languages
batch	bat, sh	commands to the command interpreter
markup	xml, html, tex	textual data, documents
word processor	xm1, rtf, docx	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	gif, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	rar, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, mp3, mp4, avi	binary file containing audio or A/V information

Figure 5.1 Common file types.

File Structure

- File types also can be used to indicate the internal structure of the file. Source and object files have structures that match the expectations of the programs that read them.
- For example, the operating system requires that executable files have a specific structure so that it can determine where in memory to load the file and what the location of the first instruction is.
- This point brings us to one of the disadvantages of having the operating system support multiple file structures. If the operating system defines five different file structures, it needs to contain the code to support these file structures.
- When new applications require information structured in ways not supported by the operating system, severe problems may result.

Internal File Structure

- Disk systems typically have a well-defined block size. All disk I/O is performed in units of one block(physical record), and all blocks are of the same size. It is unlikely that the physical record size will exactly match the length of the desired logical record.

- Logical records may even vary in length. Packing a number of logical records into physical blocks is a common solution to this problem.
- The logical record size, physical block size, and packing technique determine how many logical records are in each physical block. The packing can be done either by the user's application program or by the operating system.

5.2 Access Methods

Types of access methods are:

1. *Sequential Access*
2. *Direct Access*

1. Sequential Access

- The simplest access method is sequential access. Information in the file is processed in order, one record after the other. This mode of access is by far the most common; for example, editors and compilers usually access files in this fashion.
- A read operation-*read next*-reads the next portion of the file and automatically advances a file pointer. Similarly, the write operation-*write next*-appends to the end of the file and advances to the end of the newly written material.

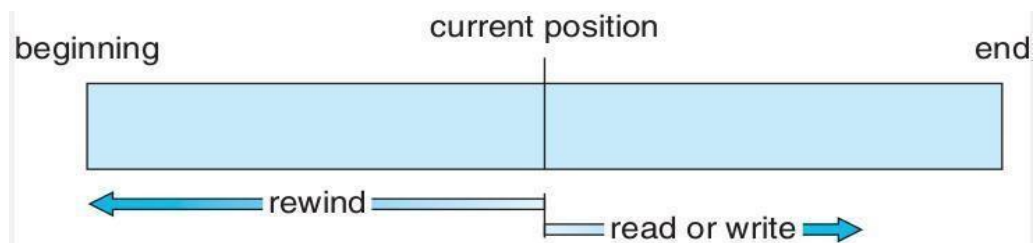


Figure 5.2 Sequential-access file.

2. Direct Access

- A file is made up of fixed length logical records that allow programs to read and write records rapidly in no particular order.
- For direct access, the file is viewed as a numbered sequence of blocks or records. Thus, we may read block 14, then read block 53, and then write block 7. There are no restrictions on the order of reading or writing for a direct-access file.
- For the direct-access method, the file operations must be modified to include the block number as a parameter. Thus, we have *read n*, where *n* is the block number, rather than *read next*, and *write n* rather than *write next*.

sequential access	implementation for direct access
reset	cp = 0;
read_next	read cp ; cp = cp + 1;
write_next	write cp; cp = cp + 1;

Figure 5.3 Simulation of sequential access on a direct-access file.

- Assuming we have a logical record of length *L*, the request for record *N* is turned into an I/O request for *L* bytes starting at location $L * (N)$ within the file (assuming the first record is *N* = 0).

3. Other Access Methods

- Other access methods can be built on top of a direct-access method. These methods generally involve the construction of an index for the file.
- To find a record in the file, we first search the index and then use the pointer to access the file directly and to find the desired record.
- For example, a retail-price file might list the universal product codes (UPCs) for items, with the associated prices. Each record consists a 10-digit UPC and a 6-digit price, for a 16-byte record
- If our disk has 1,024 bytes per block we can store 64 records per block. A file of 1,28,000 records would occupy about 2,000 blocks.
- By keeping the file sorted by UPC, we can define an index consisting of the first UPC in each block. This index would have entries of 10 digits each, or 20,000 bytes, and thus could be kept in memory.

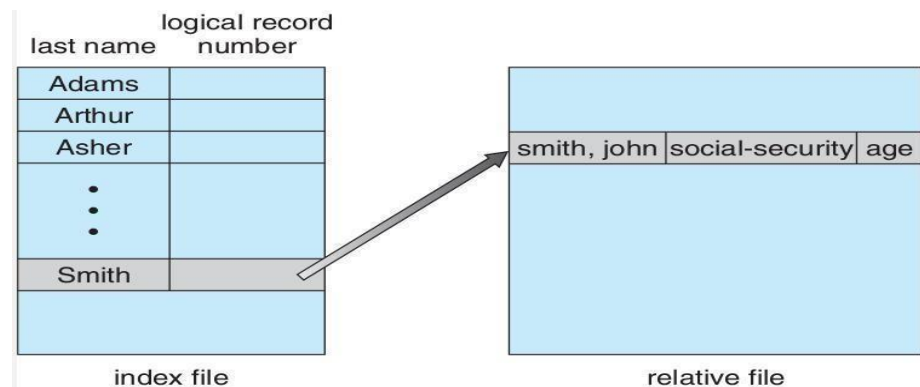


Figure 5.4 Example of index and relative files.

5.3 Directory and Disk Structure

A **directory** is a unique type of file that contains only the information needed to access files or other directories. As a result, a directory occupies less space than other types of files. File systems consist of groups of directories and the files within the directories.

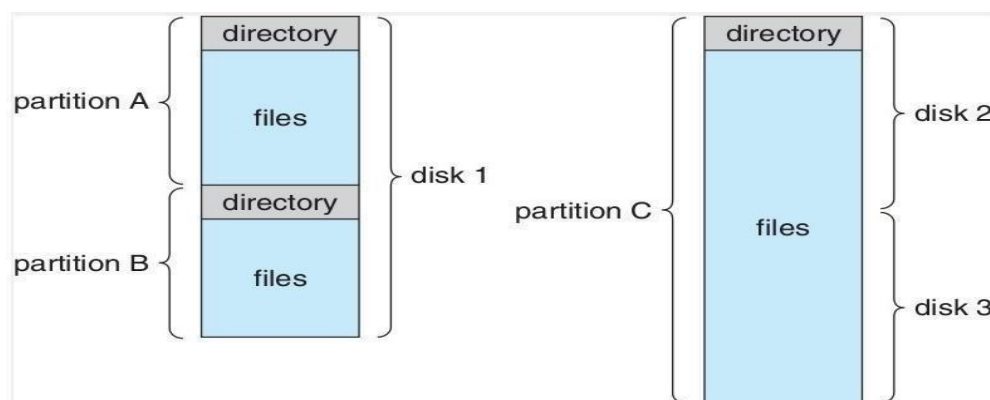


Figure 5.5 A typical file-system organization.

- Disk can be subdivided into **partitions**
- Disks or partitions can be **RAID** protected against failure
- Disk or partition can be used **raw** – without a file system, or **formatted** with a file system
- Partitions also known as minidisks, slices
- Entity containing file system known as a **volume**

- Each volume containing file system also tracks that file system's info in **device directory** or **volume table of contents**
- As well as **general-purpose file systems** there are many **special-purpose file systems**, frequently all within the same operating system or computer
- Partitioning is useful for limiting the sizes of individual file systems, putting multiple file-system types on the same device, or leaving part of the device available for other uses.
- A file system can be created on each of these parts of the disk. Any entity containing a file system is generally known as a volume.
- The volume may be a subset of a device, a whole device, or multiple devices linked together. Each volume can be thought of as a virtual disk.
-

5.3.1 Directory Overview

- The directory can be viewed as a symbol table that translates file names into their directory entries.
- We want to be able to insert entries, to delete entries, to search for a named entry, and to list all the entries in the directory.

operations that are to be performed on a directory:

- Search for a file. We need to be able to search a directory structure to find the entry for a particular file.
- Create a file. New files need to be created and added to the directory.
- Delete a file. When a file is no longer needed, we want to be able to remove it from the directory.
- List a directory. We need to be able to list the files in a directory and the contents of the directory entry for each file in the list.
- Rename a file. Because the name of a file represents its contents to its users, we must be able to change the name when the contents or use of the file changes.
- Traverse the file system. We may wish to access every directory and every file within a directory structure.

Logical Structure

- **Single-Level Directory**
- **Two-Level Directory**
- **Tree-Structured Directories**

Single-level Directory

- The simplest directory structure is the single-level directory. All files are contained in the same directory.
- A single-level directory has significant limitations, however, when the number of files increases or when the system has more than one user. Since all files are in the same directory, they must have unique names.
- Even a single user on a single-level directory may find it difficult to remember the names of all the files as the number of files increases.

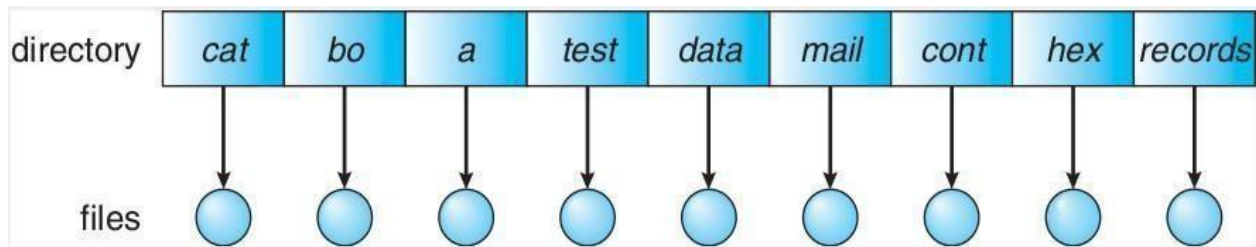


Figure 5.6 Single-level directory.

Two-Level Directory

- In the two-level directory structure, each user has his own user file directory(UFD). The UFDs have similar structures, but each lists only the files of a single user.
- When a user job starts or a user logs in, the system's master file directory(MFD) is searched. The MFD is indexed by user name or account number, and each entry points to the UFD for that user.
- When a user refers to a particular file, only his own UFD is searched. Thus, different users may have files with the same name, as long as all the file names within each UFD are unique.
- Although the two-level directory structure solves the name-collision problem, it still has disadvantages. This structure effectively isolates one user from another. To name a particular file uniquely in a two-level directory, we must give both the user name and the file name.

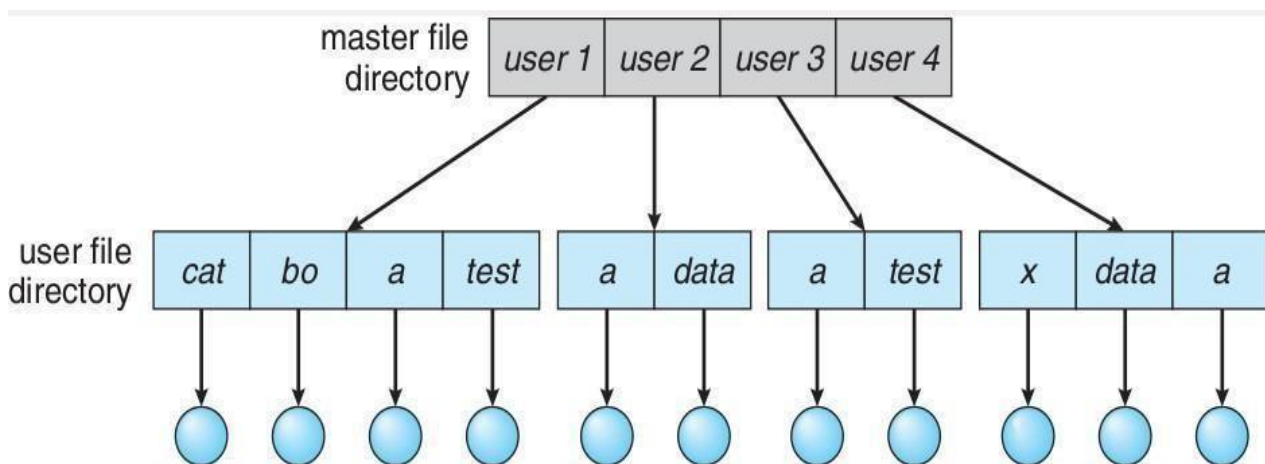


Figure 5.7 Two-level directory structure.

- For example, if user A wishes to access her own test file named test, she can simply refer to test. To access the file named test of user B (with directory-entry name userb), however, she might have to refer to /userb/test.
- Many command interpreters simply treat commands as the name of a file to load and execute. With the directory system defined presently, this file name would be searched for in the current UFD.
- One solution would be to copy the system files into each UFD. However, copying all the system files would waste an enormous amount of space.
- A special user directory is defined to contain the system files. Whenever a file name is given to be loaded, the operating system first searches the local UFD.
- If the file is found, it is used. If it is not found, the system automatically searches the special user directory that contains the system files.

Tree-Structured Directories

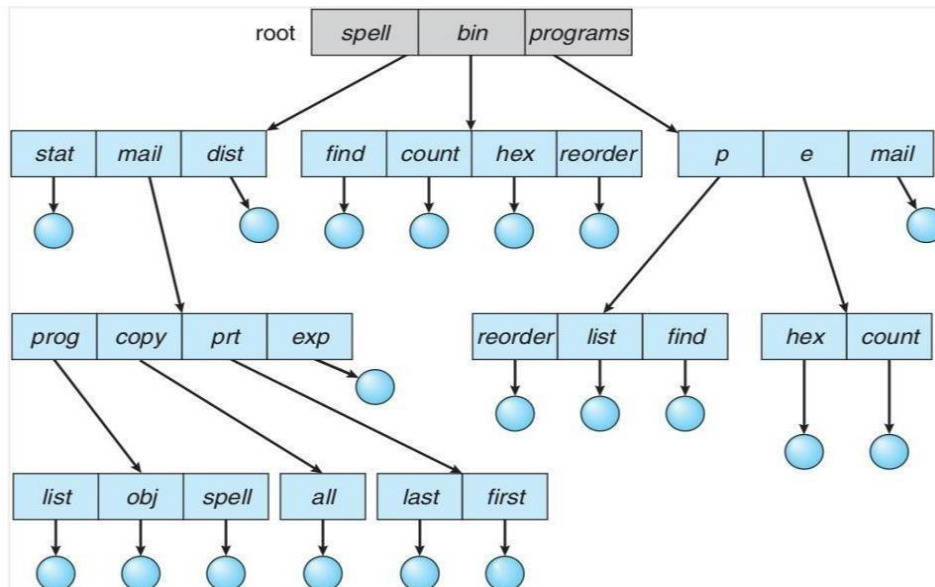


Figure 5.8 Tree-structured directory structure.

- A directory (or subdirectory) contains a set of files or subdirectories. A directory is simply another file, but it is treated in a special way.
- All directories have the same internal format. One bit in each directory entry defines the entry as a file (0) or as a subdirectory (1).
- In normal use, each process has a current directory. The current directory should contain most of the files that are of current interest to the process.
- When reference is made to a file, the current directory is searched. If a file is needed that is not in the current directory, then the user usually must either specify a path name or change the current directory to be the directory holding that file.
- Path names can be of two types: absolute and relative. An absolute path name begins at the root and follows a path down to the specified file, giving directory names on the path.
- A relative path name defines a path from the current directory. For example, in the above Drawing, if the current directory is root/spell/mail, then the relative path name prt/first refers to the same file as does the absolute path name root/spell/mail/prt/first.
- An interesting policy decision in a tree-structured directory concerns how to handle the deletion of a directory. If a directory is empty, its entry in the directory that contains it can simply be deleted.
- However, suppose the directory to be deleted is not empty but contains several files or subdirectories. The approach taken by the UNIX rm command, is to provide an option: when a request is made to delete a directory, directory's files and subdirectories are also to be deleted.

Acyclic-Graph Directories

- A tree structure prohibits the sharing of files or directories. An acyclic graph that is, a graph with no cycles-allows directories to share subdirectories and files. The same file or subdirectory may be in two different directories.
- With a shared file, only one actual file exists, so any changes made by one person are immediately visible to the other. A new file created by one person will automatically appear in all the shared subdirectories.

- Shared files and subdirectories can be implemented in several ways. A link is effectively a pointer to another file or subdirectory.
- For example, a link may be implemented as an absolute or a relative path name. When a reference to a file is made, we search the directory. If the directory entry is marked as a link, we resolve the link by using that path name to locate the real file.
- In a system where sharing is implemented by symbolic links, the deletion of a link need not affect the original file; only the link is removed.
- If the file entry itself is deleted, the space for the file is deallocated, leaving the links dangling. We can leave the links until an attempt is made to use them. At that time, we can determine that the file with name given by the link does not exist and delete the link.
- Another approach to deletion is to preserve the file until all references to it are deleted. We could keep a list of references to a file.
- When a link is established, a new entry is added to the file-reference list. When a link, we remove its entry on the list. The file is deleted when its file-reference list is empty.

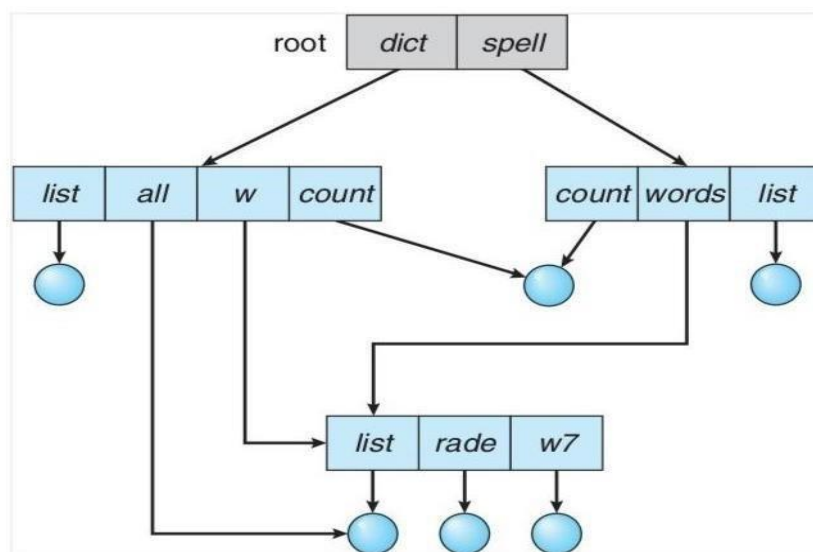


Figure 5.9 Acyclic-graph directory structure.

General Graph Directory

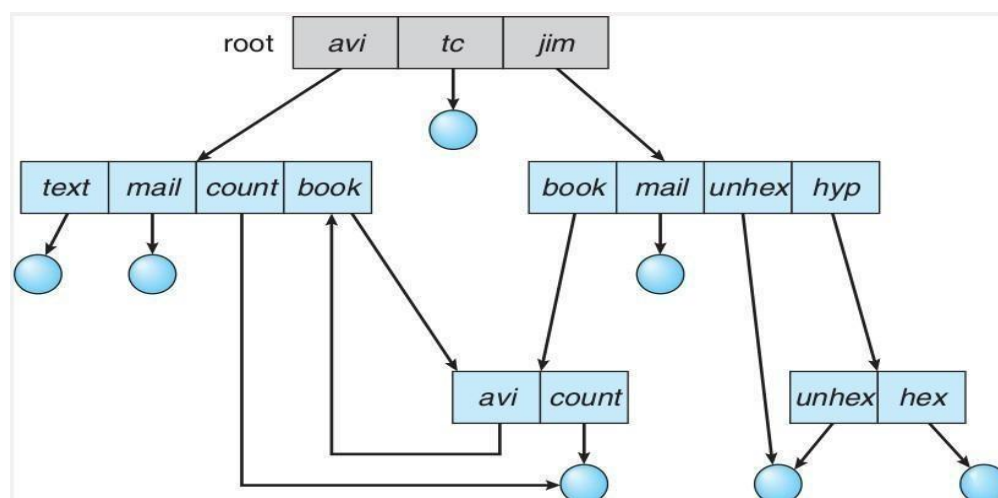


Figure 5.10 General graph directory

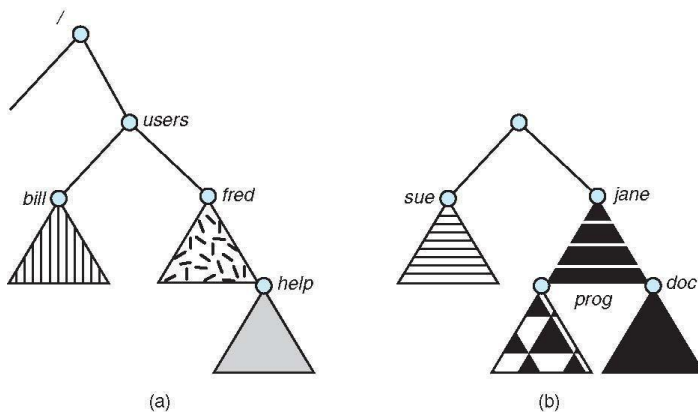
- A serious problem with using an acyclic-graph structure is ensuring that there are no cycles. The primary advantage of an acyclic graph is the relative simplicity of the algorithms to traverse

the graph and to determine when there are no more references to a file.

- With acyclic-graph directory structures, a value of 0 in the reference count means that there are no more references to the file or directory, and the file can be deleted. However, when cycles exist, the reference count may not be 0 even when it is no longer possible to refer to a directory or file. This anomaly results from the possibility of self-referencing.

5.4 File System Mounting

- Just as a file must be opened before it is used, a file system must be mounted before it can be available to processes on the system.
- The mount procedure is straightforward. The operating system is given the name of the device and the mount point—the location within the file structure where the file system is to be attached.
- Typically, a mount point is an empty directory. For instance, on a UNIX system, a file system containing a user's home directories might be mounted as /home.
- Next, the operating system verifies that the device contains a valid file system. It does so by asking the device driver to read the device directory and verifying that the directory has the expected format.
- Finally, the operating system notes in its directory structure that a file system is mounted at the specified mount point.
- To illustrate file mounting, consider the file system depicted in Drawing below, where the triangles represent subtrees. Drawing 'a' shows an existing file system, while drawing 'b' shows an unmounted volume residing on /device/disk.
- At this point, only the files on the existing file system can be accessed. Drawing 'c' shows the effects of mounting the volume residing on /device/disk over /users.



5.11 File system. (a) Existing system. (b) Unmounted volume.

Figure

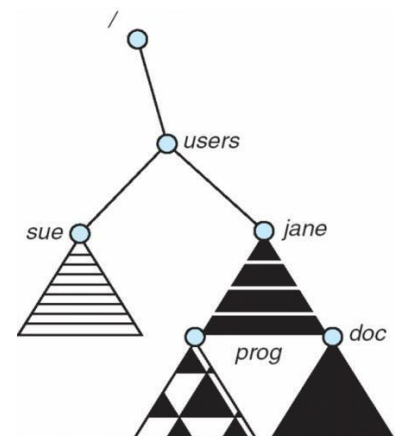


Figure 5.12 Mount point.

5.5 File Sharing

5.5.1 Multiple Users

- When an operating system accommodates multiple users, the issues of file sharing, file naming, and file protection become preeminent.
- To implement sharing and protection, the system must maintain more file and directory attributes than are needed on a single-user system.
- Most systems have evolved to use the concepts of file (or directory) owner (or user) and group. The owner is the user who can change attributes and grant access and who has the most control over the file.
- Exactly which operations can be executed by group members and other users is defined by the file's owner. The owner and group IDs of a given file (or directory) are stored with the other file attributes.

- When a user requests an operation on a file, the user ID can be compared with the owner attribute to determine if the requesting user is the owner of the file. Likewise, the group IDs can be compared.
- The result indicates which permissions are applicable. The system then applies those permissions to the requested operation and allows or denies it.

5.5.2 Remote File Systems

- The first implemented method involves manually transferring files between machines via programs like FTP.
- The second major method uses a distributed file system (DFS) in which remote directories are visible from a local machine.
- In some ways, the third method, the world wide web is a reversion to the first. A browser is needed to gain access to the remote files, and separate operations (essentially a wrapper for FTP) are used to transfer files.

The Client-Server Model

- Remote file systems allow a computer to mount one or more file systems from one or more remote machines. In this case, the machine containing the files is the server, and the machine seeking access to the files is the client.
- A server can serve multiple clients, and a client can use multiple servers, depending on the implementation details of a given client-server facility.
- Once the remote file system is mounted, file operation requests are sent on behalf of the user across the network to the server via the DFS protocol.
- Typically, a file-open request is sent along with the ID of the requesting user. The server then applies the standard access checks to determine if the user has credentials to access the file in the mode requested.
- The request is either allowed or denied. If it is allowed, a file handle is returned to the client application, and the application then can perform read, write, and other operations on the file.

Distributed Information Systems

- To make client-server systems easier to manage, distributed information systems, also known as distributed naming services, provide unified access to the information needed for remote computing.
- The domain name system(DNS) provides host-name-to-network-address translations for the entire Internet.
- The industry is moving toward use of the lightweight directory access protocol(LDAP) as a secure distributed naming mechanism.
- Sun Microsystems includes LDAP with the operating system and allows it to be employed for user authentication as well as system-wide retrieval of information, such as availability of printers.
- Conceivably, one distributed LDAP directory could be used by an organization to store all user and resource information for all the organization's computers.
- The result would be secure single sign in for users, who would enter their authentication information once for access to all computers within the organization.

5.6 Protection

- When information is stored in a computer system, we want to keep it safe from physical damage (the issue of reliability) and improper access (the issue of protection).
- Reliability is generally provided by duplicate copies of files. Many computers have systems programs that automatically copy disk files to tape at regular intervals to maintain a copy if a file system is accidentally destroyed.

5.6.1 Types of Access

Several different types of operations may be controlled:

- Read. Read from the file.
- Write. Write or rewrite the file.
- Execute. Load the file into memory and execute it.
- Append. Write new information at the end of the file.
- Delete. Delete the file and free its space for possible reuse.
- List. List the name and attributes of the file.

Other operations, such as renaming, copying, and editing the file, may also be controlled. For many systems, however, these higher-level functions may be implemented by a system program that uses the lower-level system calls.

Protection is provided at only the lower level. For instance, copying a file may be implemented simply by a sequence of read requests. In this case, a user with read access can also cause the file to be copied, printed, and so on.

5.6.2 Access Control

- The most common approach to the protection problem is to make access dependent on the identity of the user. Different users may need different types of access to a file or directory.
- The most general scheme to implement identity-dependent access is to associate with each file and directory an access control list (ACL) specifying user names and the types of access allowed for each user.
- When a user requests access to a particular file, the operating system checks the access list associated with that file. If that user is listed for the requested access, the access is allowed. Otherwise, a protection violation occurs, and the user job is denied access to the file.
- The main problem with access lists is their length. If we want to allow everyone to read a file, we must list all users with read access.

To condense the length of the access-control list, many systems recognize three classifications of users in connection with each file:

- Owner. The user who created the file is the owner.
- Group. A set of users who are sharing the file and need similar access is a group, or work group.
- Universe. All other users in the system constitute the universe.

The most common recent approach is to combine access-control lists with the more general owner, group, and universe access-control scheme just described.

For example, the UNIX system defines three fields of 3 bits each -rwx, where r controls read access, w controls write access, and x controls execution. A separate field is kept for the file owner, for the file's group, and for all other users.

IMPLEMENTATION OF FILE SYSTEM

5.7 File System Structure

- Disks provide the bulk of secondary storage on which a file system is maintained. They have two characteristics that make them a convenient medium for storing multiple files:
A disk can be rewritten in place; it is possible to read a block from the disk, modify the block, and write it back into the same place.
A disk can access directly any block of information it contains. Thus, it is simple to access any file either sequentially or randomly.
- A file system poses two quite different design problems.
 1. The first problem is defining how the file system should look to the user. This task involves defining a file and its attributes, the operations allowed on a file, and the directory structure for organizing files.
 2. The second problem is creating algorithms and data structures to map the logical file system onto the physical secondary-storage devices.

5.7.1 Layered File System

The file system itself is generally composed of many different levels. The structure shown in Drawing below.

- The lowest level, the **I/O control**, consists of device drivers and interrupt handlers to transfer information between the main memory and the disk system.
- A **device driver** can be thought of as a translator. Its input consists of high-level commands such as "retrieve block 123." Its output consists of low-level, hardware-specific instructions that are used by the hardware controller.
- The **basic file system** needs only to issue generic commands to the appropriate device driver to read and write physical blocks on the disk.
- The **file-organization module** knows about files and their logical blocks, as well as physical blocks. By knowing the type of file allocation used and the location of the file, the file-organization module can translate logical block addresses to physical block addresses.

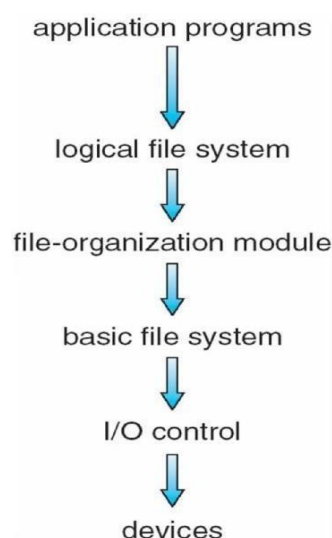


Figure 5.13 Layered file system.

- The **file-organization module** also includes the free-space manager, which tracks unallocated blocks and provides these blocks to the file-organization module when requested.
- Finally, the **logical file system** manages metadata information. Metadata includes all of the file-system structure except the actual data.
- The logical file system maintains file structure via file-control blocks. A file-control block contains information about the file, including ownership, permissions, and location of the file contents.

5.8 File System Implementation

5.8.1 Overview

Several **on-disk** and **in-memory** structures are used to implement a file system.

- **On disk**, the file system may contain information about how to boot an operating system stored there, the total number of blocks, the number and location of free blocks, the directory structure, and individual files.
 - A boot control block (per volume) can contain information needed by the system to boot an operating system from that volume. If the disk does not contain an operating system, this block can be empty. It is typically the first block of a volume.
 - A volume control block (per volume) contains volume (or partition) details, such as the number of blocks in the partition, the size of the blocks, a free-block count and free-block pointers.

A directory structure (per file system) is used to organize the files. In UFS, this includes file names and numbers.

A per-file FCB contains many details about the file. It has a unique identifier number to allow association with a directory entry.
- The **in-memory** information is used for both file-system management and performance improvement via caching. The data are loaded at mount time, updated during file-system operations, and discarded at dismount. Several types of structures may be included.

An in-memory mount table contains information about each mounted volume.

An in-memory directory-structure cache holds the directory information of recently accessed directories.

The system-wide open-file table contains a copy of the FCB of each open file, as well as other information.

- a. The per-process open-file table contains a pointer to the appropriate entry in the system-wide open-file table, as well as other information.
- b. Buffers hold file-system blocks when they are being read from disk or written to disk.

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

Figure 5.14 A typical file-control block.

1. To create a new file, an application program calls the logical file system. The logical file system knows the format of the directory structures. To create a new file, it allocates a new FCB.
2. The system then reads the appropriate directory into memory, updates it with the new file name and FCB, and writes it back to the disk. A typical FCB is shown in Drawing above.

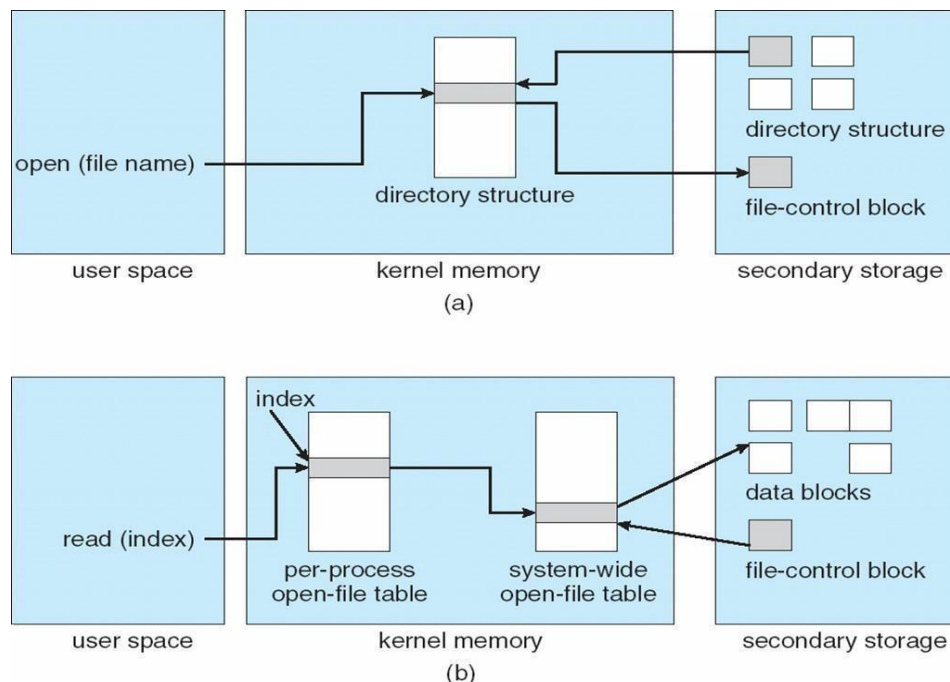


Figure 5.14 In-memory file-system structures. (a) File open. (b) File read.

- Now that a file has been created, it can be used for I/O. First, though, it must be opened. The `open()` call passes a file name to the logical file system.
- The `open()` system call first searches the system-wide open-file table to see if the file is already in use by another process. If it is, a per-process open-file table entry is created pointing to the existing system-wide open-file table.
- If the file is not already open, the directory structure is searched for the given file name. Once the file is found, the FCB is copied into a system-wide open-file table in memory.
- Next, an entry is made in the per-process open-file table, with a pointer to the entry in the system-wide open-file table and some other fields.
- The `open()` call returns a pointer to the appropriate entry in the per-process file-system table. All file operations are then performed via this pointer.

- When a process closes the file, the per-process table entry is removed, and the system-wide entry's open count is decremented.
- When all users that have opened the file close it, any updated metadata is copied back to the disk-based directory structure, and the system-wide open-file table entry is removed.

Partitions and Mounting

- A disk can be sliced into multiple partitions, or a volume can span multiple partitions on multiple disks. Each partition can be either "raw," containing no file system, or "cooked," containing a file system.
- Boot information can be stored in a separate partition. Again, it has its own format, because at boot time the system does not have the file-system code loaded and therefore cannot interpret the file-system format.
- Rather, boot information is usually a sequential series of blocks, loaded as an image into memory. Execution of the image starts at a predefined location, such as the first byte.
- This boot loader in turn knows enough about the file-system structure to be able to find and load the kernel and start it executing.
- The root partition which contains the operating-system kernel and some- times other system files, is mounted at boot time.
- Other volumes can be automatically mounted at boot or manually mounted later, depending on the operating system.
- As part of a successful mount operation, the operating system verifies that the device contains a valid file system. It does so by asking the device driver to read the device directory and verifying that the directory has the expected format.

5.9 Virtual File Systems

An obvious but suboptimal method of implementing multiple types of file systems is to write directory and file routines for each type.

- Instead, however, most operating systems, including UNIX, use object-oriented techniques to simplify, organize, and modularize the implementation.
- The use of these methods allows very dissimilar file-system types to be implemented within the same structure, including network file systems, such as NFS.
- The file-system implementation consists of three major layers, as depicted schematically in Drawing below. The first layer is the file-system interface, based on the `open()`, `read()`, `write()`, and `close()` calls and on file descriptors.
- The second layer is called the virtual file system(VFS) layer. VFS layer serves two important functions:
- It separates file-system-generic operations from their implementation by defining a clean VFS interface. Several implementations for the VFS interface may coexist on the same machine, allowing transparent access to different types of file systems mounted locally.

It provides a mechanism for uniquely representing a file throughout a network. The VFS is based on a file-representation structure, called a vnode that contains a numerical designator for a network-wide unique file. The VFS activates file-system-specific operations to handle local requests according to their file-system types and calls the NFS protocol procedures for remote requests.

The layer implementing the file-system type or the remote-file-system protocol is the third layer of the architecture.

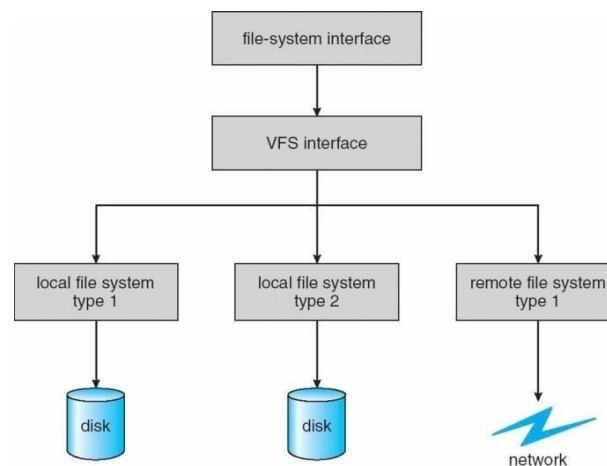


Figure 5.15 Schematic view of a virtual file system.

5.10 Directory Implementation

5.10.1 Linear List

The simplest method of implementing a directory is to use a linear list of file names with pointers to the data blocks.

- To create a new file, we must first search the directory to be sure that no existing file has the same name. Then, we add a new entry at the end of the directory.
- To delete a file, we search the directory for the named file and then release the space allocated to it.

The real disadvantage of a linear list of directory entries is that finding a file requires a linear search. Directory information is used frequently, and users will notice if access to it is slow.

A sorted list allows a binary search and decreases the average search time. However, the requirement that the list be kept sorted may complicate creating and deleting files.

5.10.2 Hash Table

Another data structure used for a file directory is a hash table. With this method, a linear list stores the directory entries, but a hash data structure is also used.

The hash table takes a value computed from the file name and returns a pointer to the file name in the linear list. Therefore, it can greatly decrease the directory search time.

The major difficulties with a hash table are its fixed size and the dependence of the hash function on that size.

For example, assume that we make a hash table that holds 64 entries. The hash function converts file names into integers from 0 to 63, for instance, by using the remainder of a division by 64.

If we later try to create a 65th file, we must enlarge the hash table—say, to 128 entries. As a result, we need a new hash function that must map file names to the range 0 to 127, and we must reorganize the existing directory entries to reflect their new hash-function values.

5.11 Allocation Methods

The direct-access nature of disks allows us flexibility in the implementation of files. The main problem is how to allocate space to these files so that disk space is utilized effectively and files can be accessed quickly. Three major methods of allocating disk space are in wide use:

1. **Contiguous Allocation**
2. **Linked Allocation**
3. **Indexed Allocation**

1. Contiguous Allocation

Contiguous Allocation requires that each file occupy a set of contiguous blocks on the disk. Contiguous allocation of a file is defined by the disk address and length of the first block.

- the file is n blocks long and starts at location b , then it occupies blocks $b, b + 1, b + 2, \dots, b + n - 1$.
- The directory entry for each file indicates the address of the starting block and the length of the area allocated for this file.
- Accessing a file that has been allocated contiguously is easy. For sequential access, the file system remembers the disk address of the last block referenced and, when necessary, reads the next block.
- For direct access to block i of a file that starts at block b , we can immediately access block $b + i$. Thus, both sequential and direct access can be supported by contiguous allocation.
- Contiguous allocation has some problems, however. One difficulty is finding space for a new file. Which involves how to satisfy a request of size n from a list of free holes.

First fit and best fit are the most common strategies used to select a free hole from the set of available holes. All these algorithms suffer from the problem of **external fragmentation**.

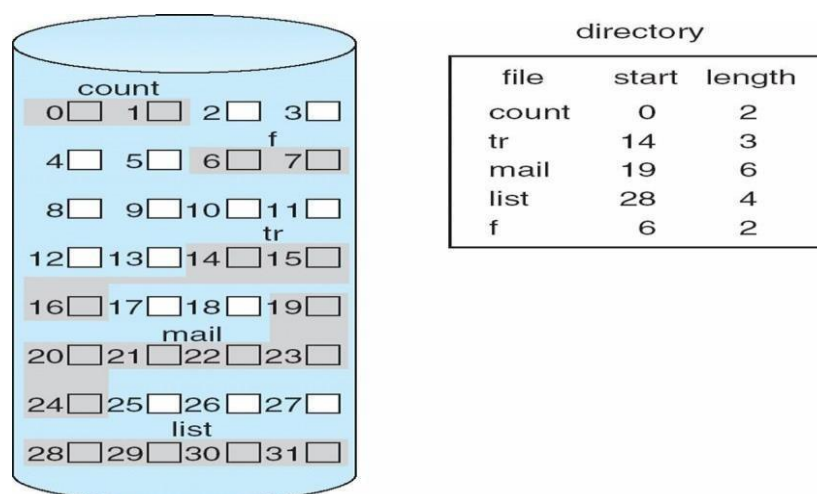


Figure 5.16 Contiguous allocation of disk space.

- One strategy for preventing loss of significant amounts of disk space to external fragmentation is

to copy an entire file system onto another disk or tape. The original disk is then freed completely, creating one large contiguous free space.

- We then copy the files back onto the original disk by allocating contiguous space from this one large hole. This scheme effectively compacts all free space into one contiguous space, solving the fragmentation problem.
- However, the cost of this compaction is time and it can be particularly severe for large hard disks that use contiguous allocation.
- Another problem with contiguous allocation is determining how much space is needed for a file. If we allocate too little space to a file, we may find that the file cannot be extended.
 - Especially with a best-fit allocation strategy, the space on both sides of the file may be in use. Hence, we cannot make the file larger in place.
- A file that will grow slowly over a long period (months or years) must be allocated enough space for its final size, even though much of that space will be unused for a long time. The file therefore has a large amount of **internal fragmentation**.

2. Linked Allocation

Linked allocation solves all problems of contiguous allocation. With linked allocation, each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk.

- The directory contains a pointer to the first and last blocks of the file. For example, a file of five blocks might start at block 9 and continue at block 16, then block 1, then block 10, and finally block 25 (Drawing below).
- Each block contains a pointer to the next block. These pointers are not made available to the user. Thus, if each block is 512 bytes in size, and a disk address (the pointer) requires 4 bytes, then the user sees blocks of 508 bytes.
- To create a new file, we simply create a new entry in the directory. With linked allocation, each directory entry has a pointer to the first disk block of the file. This pointer is initialized to nil to signify an empty file. The size field is also set to 0.
- A write to the file causes the free-space management system to find a free block, and this new block is written to and is linked to the end of the file.
- To read a file, we simply read blocks by following the pointers from block to block. There is no external fragmentation with linked allocation, and any free block on the free-space list can be used to satisfy a request. A file can continue to grow as long as free blocks are available.

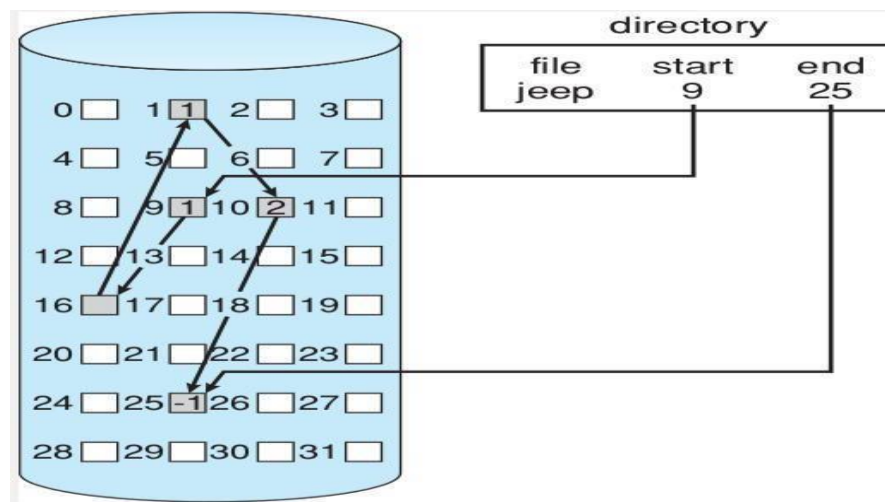


Figure 5.17 Linked allocation of disk space.

- Linked allocation does have disadvantages, however. The major problem is that it can be used effectively only for sequential-access files.
- Another disadvantage is the space required for the pointers. If a pointer requires 4 bytes out of a 512-byte block, then 0.78 percent of the disk is being used for pointers, rather than for information.
- The usual solution to this problem is to collect blocks into multiples, called clusters and to allocate clusters rather than blocks. Pointers then use a much smaller percentage of the file's disk space.
- The cost of this approach is an increase in internal fragmentation, because more space is wasted when a cluster is partially full than when a block is partially full.
- An important variation on linked allocation is the use of a file allocation table(FAT). A section of disk at the beginning of each volume is set aside to contain the table. The table has one entry for each disk block and is indexed by block number.
- The FAT is used in much the same way as a linked list. The directory entry contains the block number of the first block of the file. The table entry indexed by that block number contains the block number of the next block in the file.
- This chain continues until it reaches the last block, which has a special end-of-file value as the table entry. An unused block is indicated by a table value of 0.
- Allocating a new block to a file is a simple matter of finding the first 0-valued table entry and replacing the previous end-of-file value with the address of the new block. The 0 is then replaced with the end-of-file value.

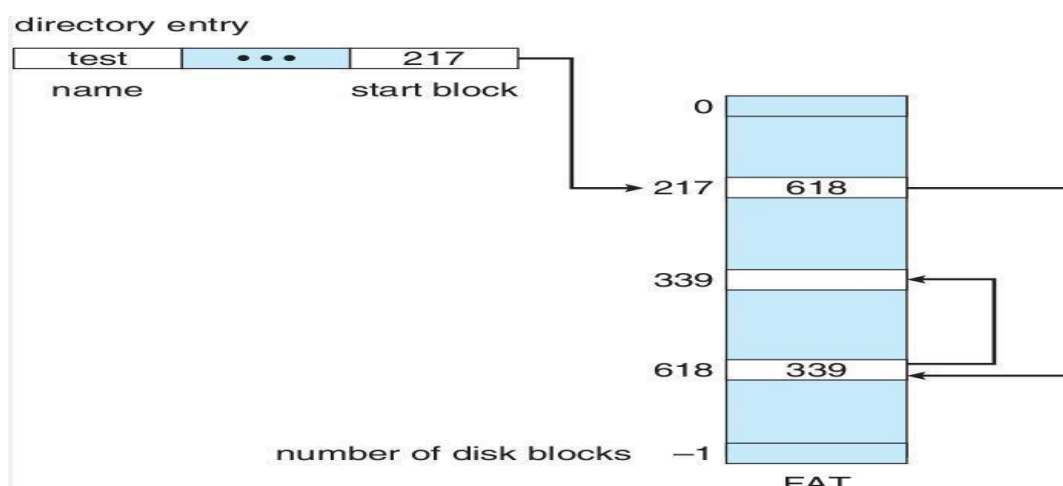


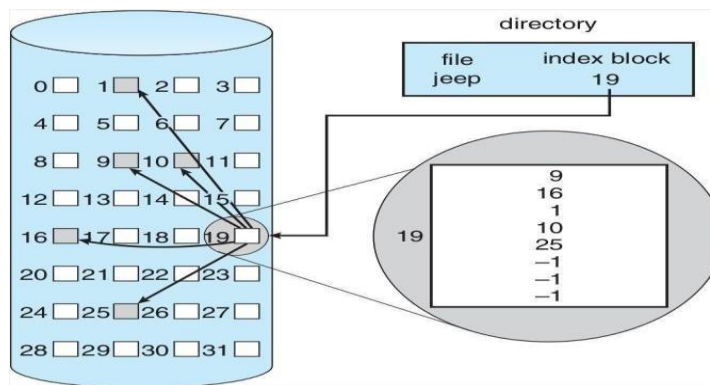
Figure 5.18 File-allocation table.

3. Indexed Allocation

Linked allocation solves the external-fragmentation and size-declaration problems of contiguous allocation. However, in the absence of a FAT, linked allocation cannot support efficient direct access.

Indexed allocation solves this problem by bringing all the pointers together into one location:

- The index block. Each file has its own index block, which is an array of disk-block addresses. The i^{th} entry in the index block points to the i^{th} block of the file.
- The directory contains the address of the index block. To find and read the i^{th} block, we use the pointer in the i^{th} index-block entry. When the file is created, all pointers in the index block are set to nil.
- When the i^{th} block is first written, a block is obtained from the free-space management and its address is put in the i^{th} index-block entry.
- Indexed allocation supports direct access, without suffering from external fragmentation, because any free block on the disk can satisfy a request for more space.

**Figure 5.19** Indexed allocation of disk space.

- Every file must have an index block, so we want the index block to be as small as possible. If the index block is too small, however, it will not be able to hold enough pointers for a large file. Mechanisms for this purpose include the following:
- **Linked scheme.** An index block is normally one disk block. Thus, it can be read and written directly by itself. To allow for large files, we can link together several index blocks.
- **Multilevel index.** A variant of linked representation uses a first-level index block to point to a set of second-level index blocks, which in turn point to the file blocks. To access a block, the operating system uses the first-level index to find a second-level index block and then uses that block to find the desired data block.
indirect block.

Performance

- The allocation methods that we have discussed vary in their storage efficiency and data-block access times. Both are important criteria in selecting the proper method an operating system to implement.
- A system with mostly sequential access should not use the same method as a system with mostly random access.
- For any type of access, contiguous allocation requires only one access to get a disk block. Since we can easily keep the initial address of the file in memory, we can calculate immediately the disk address of the i^{th} block and read it directly.
- For linked allocation, we also keep the address of the next block in memory. This method is fine

for sequential access; for direct access, however, an access to the i^{th} block might require i disk reads.

- Some systems support direct-access files by using contiguous allocation and sequential-access files by using linked allocation.
- A file created for sequential access will be linked and cannot be used for direct access. A file created for direct access will be contiguous and can support both direct access and sequential access, but its maximum length must be declared when it is created.
- Indexed allocation is more complex. If the index block is already in memory, then the access can be made directly. However, keeping the index block in memory requires considerable space.
- Some systems combine contiguous allocation with indexed allocation by using contiguous allocation for small files (up to three or four blocks) and automatically switching to an indexed allocation if the file grows large.
- Indexed more complex
 - Single block access could require 2 index block reads then data block read
 - Clustering can help improve throughput, reduce CPU overhead
- Adding instructions to the execution path to save one disk I/O is reasonable
 - Intel Core i7 Extreme Edition 990x (2011) at 3.46Ghz = 159,000 MIPS
 - http://en.wikipedia.org/wiki/Instructions_per_second
 - Typical disk drive at 250 I/Os per second
 - $159,000 \text{ MIPS} / 250 = 630$ million instructions during one disk I/O
 - Fast SSD drives provide 60,000 IOPS
 - $159,000 \text{ MIPS} / 60,000 = 2.65$ millions instructions during one disk I/O
 -

5.12 Free Space Management

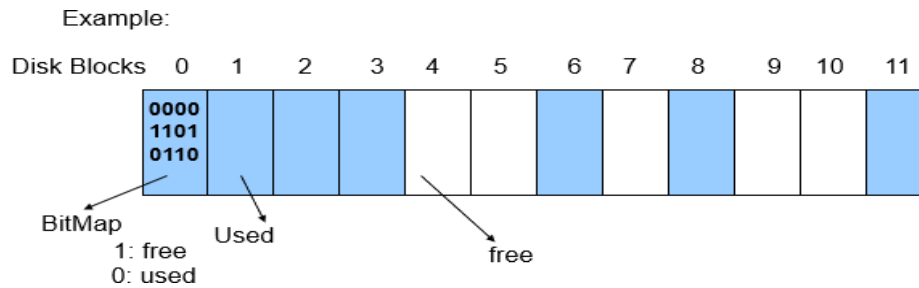
- Since disk space is limited, we need to reuse the space from deleted files for new files. To keep track of free disk space, the system maintains a free-space list.
- The free-space list records all free disk blocks-those not allocated to some file or directory. To create a file, we search the free-space list for the required amount of space and allocate that space

to the new file. This space is then removed from the free-space list.

1. Bit Vector

- Frequently, the free-space list is implemented as a bit map or bit vector. Each block is represented by 1 bit. If the block is free, the bit is 1; if the block is allocated, the bit is 0.
- For example, consider a disk where blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 are free and the rest of the blocks are allocated. The free-space bit map would be

$$001111001111110001100000011100000 \dots$$
- The main advantage of this approach is its relative simplicity and its efficiency in finding the first free block or n consecutive free blocks on the disk.
- One technique for finding the first free block on a system that uses a bit-vector to allocate disk space is to sequentially check each word in the bit map to see whether that value is not 0, since a 0-valued word contains only 0 bits and represents a set of allocated blocks.
- The first non-0 word is scanned for the first 1 bit, which is the location of the first free block. The calculation of the block number is



2. Linked List

- Another approach to free-space management is to link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory.
- This first block contains a pointer to the next free disk block, and so on. Recall our earlier example, in which blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 were free and the rest of the blocks were allocated.
- In this situation, we would keep a pointer to block 2 as the first free block. Block 2 would contain a pointer to block 3, which would point to block 4, which would point to block 5, which would point to block 8, and so on.

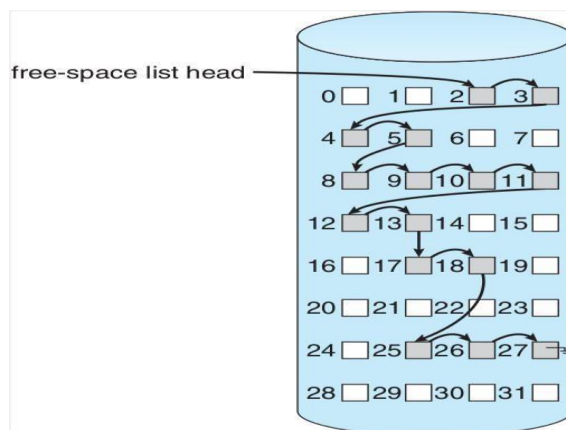
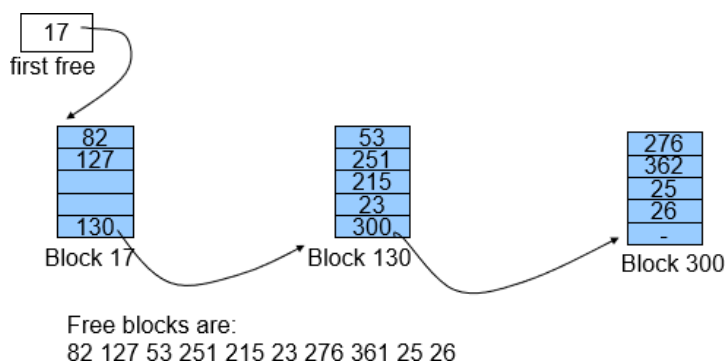


Figure Linked free-space list on disk

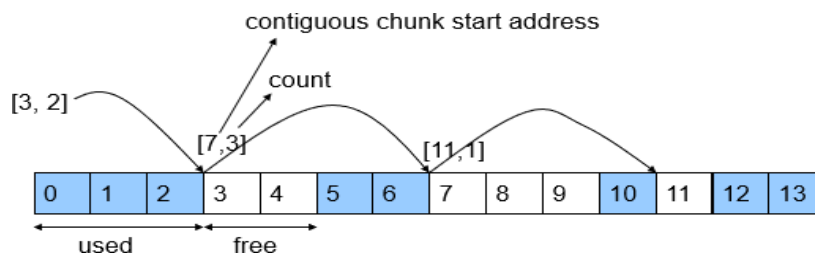
3. Grouping

- A modification of the free-list approach stores the addresses of n free blocks in the first free block. The first $n-1$ of these blocks are actually free.
- The last block contains the addresses of another n free blocks, and so on. The addresses of a large number of free blocks can now be found quickly.



4. Counting

- Another approach takes advantage of the fact that, generally, several contiguous blocks may be allocated or freed simultaneously, particularly when space is allocated with the contiguous-allocation algorithm or through clustering.



- Thus, rather than keeping a list of n free disk addresses, we can keep the address of the first free block and the number (n) of free contiguous blocks that follow the first block. Each entry in the free-space list then consists of a disk address and a count.

Secondary Storage Structures, Protection

5.13 Overview of Mass Storage

Physical structures of Secondary and tertiary storage devices.

1. Magnetic Disks

2. Magnetic Tapes

5.13.1 Structure Magnetic disks

- Magnetic Disks provide the bulk of secondary storage for modern computer systems.

- Each disk **platter** has a flat circular shape, like a CD. The two surfaces of a platter are covered with a magnetic material. We store information by recording it magnetically on the platters.
- A **read-write head** "flies" just above each surface of every platter. The heads are attached to a disk arm that moves all the heads as a unit.
- The surface of a platter is logically divided into circular **tracks**, which are subdivided into **sectors**. The set of tracks that are at one arm position makes up a **cylinder**.
- When the disk is in use, a drive motor spins it at high speed. Most drives rotate 60 to 200 times per second. Disk speed has two parts. The **transfer rate** is the rate at which data flow between the drive and the computer.
- The positioning time sometimes called the random-access time consists of the time necessary to move the disk arm to the desired cylinder, called the **seek time**, and the time necessary for the desired sector to rotate to the disk head, called the **rotational latency**.
- Because the disk head flies on an extremely thin cushion of air (measured in microns), there is a danger that the head will make contact with the disk surface.
- Although the disk platters are coated with a thin protective layer, the head will sometimes damage the magnetic surface. This accident is called a **head crash** normally cannot be repaired; the entire disk must be replaced.
- A disk can be removable, allowing different disks to be mounted as needed.

Moving-head Disk Mechanism

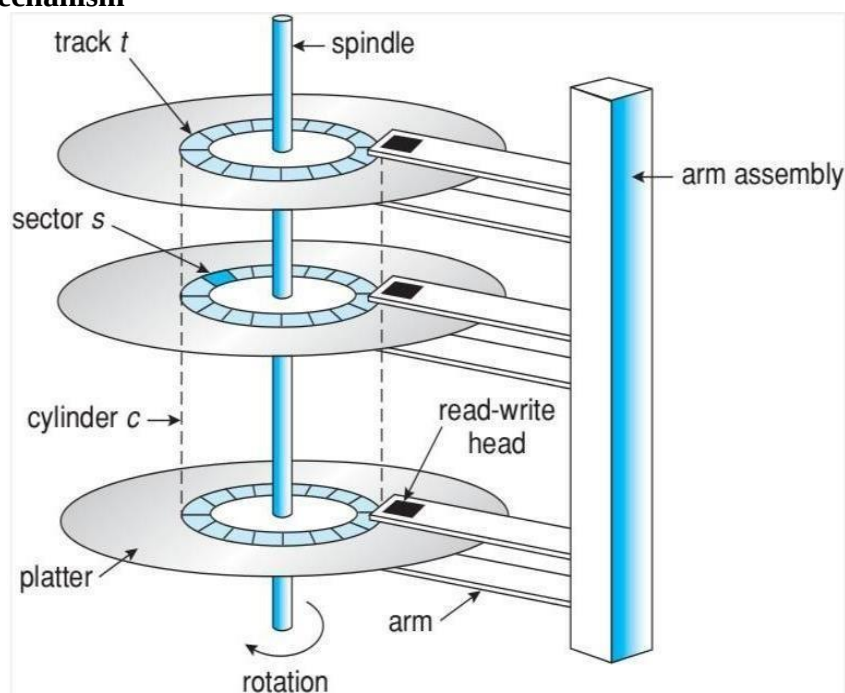


Figure 5.20 Moving-head disk mechanism.

5.13.2 Magnetic Tapes

- Magnetic Tape was used as an early secondary-storage medium. Although it is relatively permanent and can hold large quantities of data, its access time is slow compared with magnetic disk.
- In addition, random access to magnetic tape is about a thousand times slower than random access to magnetic disk, so tapes are not very useful for secondary storage.
- Tapes are used mainly for backup, for storage of infrequently used information, and as a medium for transferring information from one system to another.
- Moving to the correct spot on a tape can take minutes, but once positioned, tape drives can write

data at speeds comparable to disk drives.

- Tape capacities vary greatly, depending on the particular kind of tape drive. Typically, they store from 20GB to 200GB.

5.14 Disk Structure

- Modern disk drives are addressed as large one-dimensional arrays of logical blocks, where the logical block is the smallest unit of transfer.
- The one-dimensional array of logical blocks is mapped onto the sectors of the disk sequentially.
- Sector 0 is the first sector of the first track on the outermost cylinder. The mapping proceeds in order through that track, then through the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost.
- By using this mapping, we can at least in theory convert a logical block number into disk address that consists of a cylinder number, a track number within that cylinder, and a sector number within that track.
- In practice, it is difficult to perform this translation, for two reasons. First, most disks have some defective sectors, but the mapping hides this by substituting spare sectors from elsewhere on the disk. Second, the number of sectors per track is not a constant on some drives.
- On media that uses **constant linear velocity (CLV)** the density of bits per track is uniform. The farther a track is from the center of the disk, the greater its length, so the more sectors it can hold. As we move from outer zones to inner zones, the number of sectors per track decreases. The drive increases its rotation speed as the head moves from the outer to the inner tracks to keep the same rate of data moving under the head.

Alternatively, the disk rotation speed can stay constant; in this case, the density of bits decreases from inner tracks to outer tracks to keep the data rate constant. This method is known as **constant angular velocity (CAV)**.

5.15. Disk Scheduling

One of the responsibilities of the operating system is to use the hardware efficiently. For the disk drives, meeting this responsibility requires having fast access time and large disk bandwidth.

The access time has two major components.

- The **seek time** is the time for the disk arm to move the heads to the cylinder containing the desired sector.
- The **rotational latency** is the additional time for the disk to rotate the desired sector to the disk head.
- The disk bandwidth is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.
- We can improve both the access time and the bandwidth by managing the order in which disk I/O requests are serviced. Whenever a process needs I/O to or from the disk, it issues a system call to the operating system.
- If the desired disk drive and controller are available, the request can be serviced immediately. If the drive or controller is busy, any new requests for service will be placed in the queue of pending requests for that drive.
- For a multiprogramming system with many processes, the disk queue may often have several pending requests. Thus, when one request is completed, the operating system chooses which pending request to service next.

Disk Scheduling Algorithms in OS

1. FCFS Disk Scheduling Algorithm
2. SSTF Disk Scheduling Algorithm
3. SCAN Disk Scheduling Algorithm
4. CSCAN Disk Scheduling Algorithm
5. LOOK Disk Scheduling Algorithm
6. CLOOK Disk Scheduling Algorithm

1. FCFS

- The simplest form of disk scheduling is, of course, the first-come, first-served (FCFS) algorithm. Consider, for example, a disk queue with requests for I/O to blocks on cylinders
98, 183, 37, 122, 14, 124, 65, 67
- in that order. If the disk head is initially at cylinder 53, it will first move from 53 to 98, then to 183, 37, 122, 14, 124, 65, and finally to 67, for a total head movement of 640 cylinders.
- The wild swing from 122 to 14 and then back to 124 illustrates the problem with this schedule.

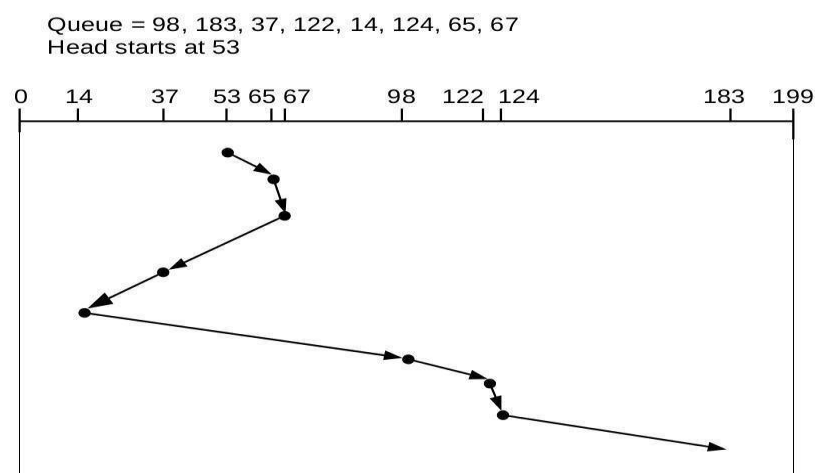


Figure 5.21 FCFS disk scheduling.

2. SSTF

- It seems reasonable to service all the requests close to the current head position before moving the head far away to service other requests.
- This assumption is the basis for the **shortest seek time first (SSTF)** algorithm. The SSTF algorithm selects the request with the least seek time from the current head position.
- Since seek time increases with the number of cylinders traversed by the head, SSTF chooses the pending request closest to the current head position.
- For our example request queue, the closest request to the initial head position (53) is at cylinder 65. Once we are at cylinder 65, the next closest request is at cylinder 67. From there, the request at cylinder 37 is closer than the one at 98, so 37 is served next. Continuing, we service the request at cylinder 14, then 98, 122, 124, and finally 183.
- This scheduling method results in a total head movement of only 236 cylinders—little more than one-third of the distance needed for FCFS scheduling of this request queue. Clearly, this algorithm gives a substantial improvement in performance.
- SSTF scheduling is essentially a form of shortest-job-first (SJF) scheduling; and like SJF scheduling, it may cause starvation of some requests.
 - At cylinder 0, the arm will reverse and will move toward the other end of the disk, servicing the requests at 65, 67, 98, 122, 124, and 183.

- If a request arrives in the queue just in front of the head, it will be serviced almost immediately; a request arriving just behind the head will have to wait until the arm moves to the end of the disk, reverses direction, and comes back.

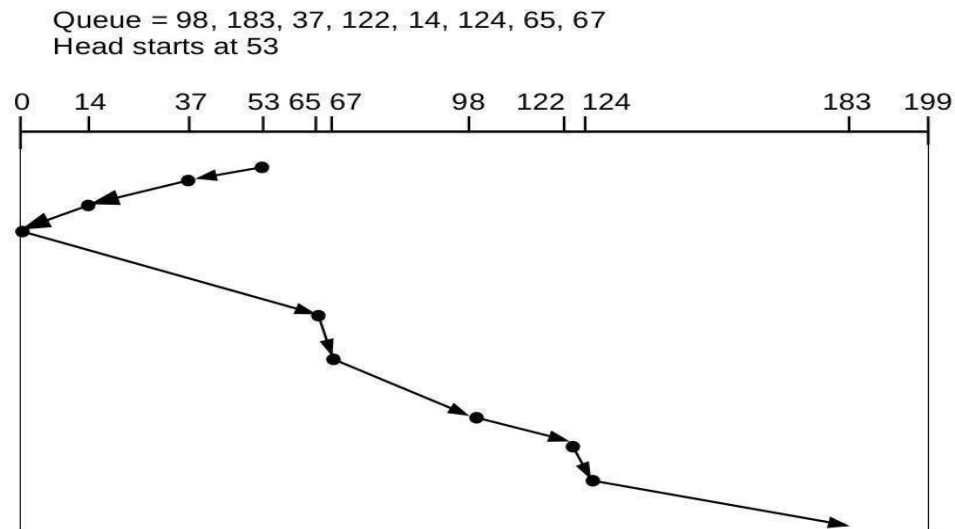


Figure 5. 22 SSTF disk scheduling.

3. SCAN

- In the SCAN algorithm disk arm starts at one end of the disk, and moves toward the other end, servicing requests as it reaches each cylinder, until it gets to the other end of the disk.
- At the other end, the direction of head movement is reversed, and servicing continues. The head continuously scans back and forth across the disk. The SCAN algorithm is sometimes called the **elevator algorithm**.
- Since the disk arm behaves just like an elevator in a building, first servicing all the requests going up and then reversing to service requests the other way.
- Before applying SCAN to schedule the requests on cylinders 98, 183, 37, 122, 14, 124, 65, and 67, we need to know the direction of head movement in addition to the head's current position.
- Assuming that the disk arm is moving toward 0 and that the initial head position is again 53, the head will next service 37 and then 14.
- At cylinder 0, the arm will reverse and will move toward the other end of the disk, servicing the requests at 65, 67, 98, 122, 124, and 183.
- If a request arrives in the queue just in front of the head, it will be serviced almost immediately; a request arriving just behind the head will have to wait until the arm moves to the end of the disk, reverses direction, and comes back.

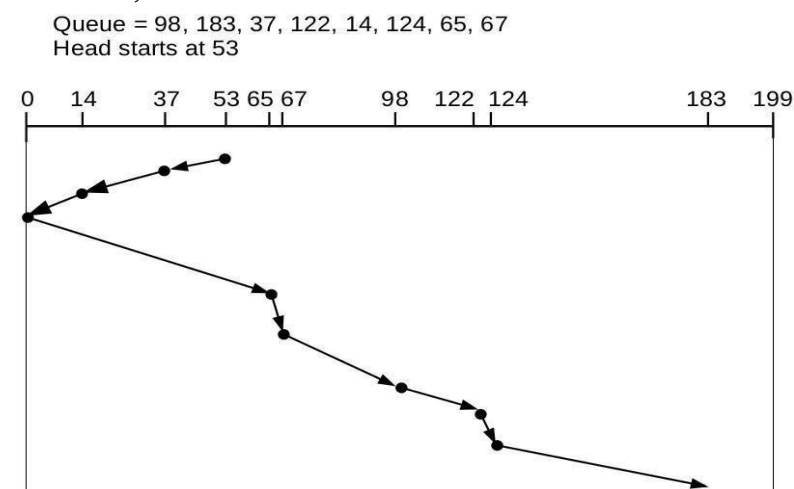


Figure 5.23 SCAN disk scheduling.

4. C-SCAN

- Circular SCAN(C-SCAN) scheduling is a variant of SCAN designed to provide a more uniform wait time. Like SCAN, C-SCAN moves the head from one end of the disk to the other, servicing requests along the way.
- When the head reaches the other end, however, it immediately returns to the beginning of the disk without servicing any requests on the return trip.

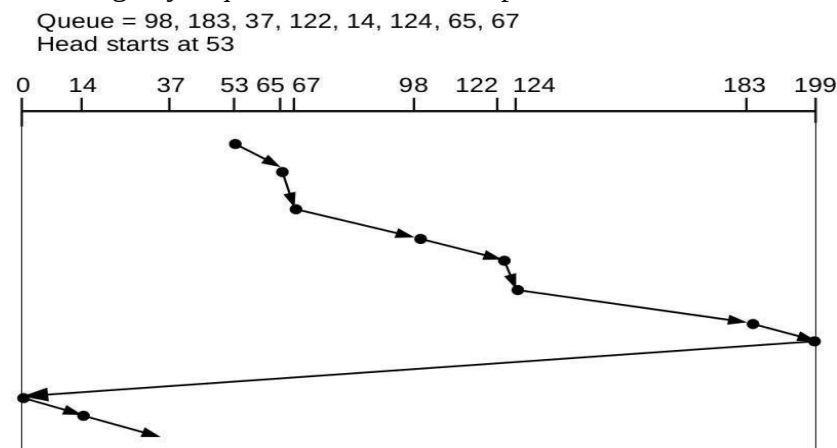


Figure 5.24 C-SCAN disk scheduling.

5. LOOK Scheduling

- Both SCAN and C-SCAN move the disk arm across the full width of the disk. More commonly the arm goes only as far as the final request in each direction.
- Then, it reverses direction immediately, without going all the way to the end of the disk. Versions of SCAN and C-SCAN that follow this pattern are called LOOK and C-LOOK scheduling, because they look for a request before continuing to move in a given direction. **C-LOOK disk scheduling** is shown below.

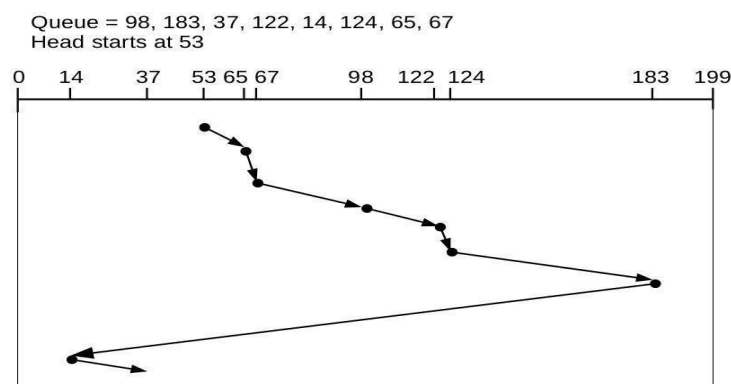


Figure 5.25 C-LOOK disk scheduling.

Selecting a Disk-Scheduling Algorithm

- SSTF is common and has a natural appeal because it increases performance over FCFS. SCAN and C-SCAN perform better for systems that place a heavy load on the disk, because they are less likely to cause a starvation problem.
- With any scheduling algorithm however, performance depends heavily on the number and types of requests.
- For instance, suppose that the queue usually has just one outstanding request. Then, all scheduling algorithms behave the same, because they have only one choice of where to move the disk head: they all behave like FCFS scheduling.
- Requests for disk service can be greatly influenced by the file-allocation method. A program

reading a contiguously allocated file will generate several requests that are close together on the disk, resulting in limited head movement.

- A linked or indexed file in contrast, may include blocks that are widely scattered on the disk, resulting in greater head movement.
- Because of these complexities, the disk-scheduling algorithm should be written as a separate module of the operating system, so that it can be replaced with a different algorithm if necessary. Either SSTF or LOOK is a reasonable choice for the default algorithm.

5.16 Disk Management

5.16.1 Disk Formatting

- A new magnetic disk is a blank slate: it is just a platter of a magnetic recording material. Before a disk can store data, it must be divided into sectors that the disk controller can read and write. This process is called or **Low-level formatting** fills the disk with a special data structure for each sector.
- The data structure for a sector typically consists of a header, a data area (usually 512 bytes in size), and a trailer. The header and trailer contain information used by the disk controller, such as a sector number and an **error correcting code (ECC)**.
- When the controller writes a sector of data during normal I/O, the ECC is updated with a value calculated from all the bytes in the data area. When the sector is read, the ECC is recalculated and compared with the stored value.
- If the stored and calculated numbers are different, this mismatch indicates that the data area of the sector has become corrupted and that the disk sector may be bad.
- Most hard disks are low-level-formatted at the factory as a part of the manufacturing process. This formatting enables the manufacturer to test the disk and to initialize the mapping from logical block numbers to defect-free sectors on the disk.
- Before system can use a disk to hold files, the operating system still needs to record its own data structures on the disk. It does so in two steps. The first step is to partition the disk into one or more groups of cylinders.
- The operating system can treat each partition as though it were a separate disk. For instance, one partition can hold a copy of the operating system's executable code, while another holds user files.
- The second step is logical formatting or creation of a file system. In this step, the operating system stores the initial file-system data structures onto the disk. These data structures may include maps of free and allocated space (a FAT or inodes) and an initial empty directory.

5.16.2 Boot Block

- For a computer to start running-for instance, when it is powered up or rebooted -it must have an initial program to run.
- This initial **bootstrap program** tends to be simple. It initializes all aspects of the system, from CPU registers to device controllers and the contents of main memory, and then starts the operating system.
- To do its job, the bootstrap program finds the operating-system kernel on disk, loads that kernel into memory, and jumps to an initial address to begin the operating-system execution.
- For most computers, the bootstrap is stored in read-only memory(ROM). The problem is that changing this bootstrap code requires changing the ROM hardware chip.

- For this reason, most systems store a tiny bootstrap loader program in the boot ROM whose only job is to bring in a full bootstrap program from disk. The full bootstrap program can then be changed easily.
- The code in the boot ROM instructs the disk controller to read the boot blocks into memory and then starts executing that code.

Bootting from a Disk in Windows

- Booting begins in a Windows 2000 system by running code that is resident in the system's ROM memory. This code directs the system to read the boot code from the MBR.
- In addition to containing boot code, the MBR contains a table listing the partitions for the hard disk and a flag indicating which partition the system is to be booted from. Once the system identifies the boot partition, it reads the first sector from that partition and continues with the remainder of the boot process.

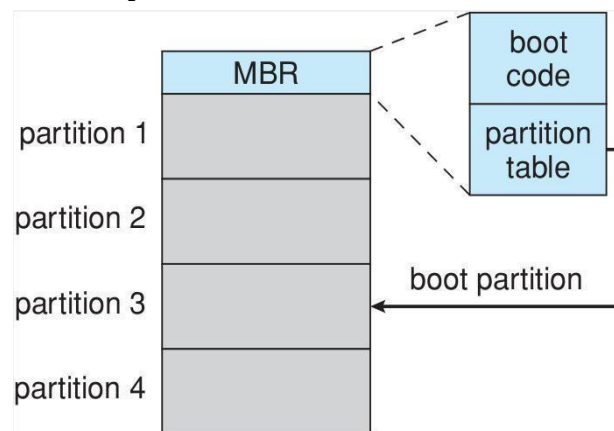


Figure 5.26 Booting from disk in Windows 2000.

Bad Blocks

- More frequently, one or more sectors become defective. Most disks even come from the factory with bad blocks. Depending on the disk and controller in use, these blocks are handled in a variety of ways.
- For instance, the MS-DOS format command performs logical formatting and, as a part of the process, scans the disk to find bad blocks. If format finds a bad block, it writes a special value into the corresponding FAT entry to tell the allocation routines not to use that block.
- If blocks go bad during normal operation, a special program must be run manually to search for the bad blocks and to lock them away. Data that resided on the bad blocks usually are lost.
- More sophisticated disks, such as the SCSI disks used in high-end PCs and most workstations and servers, are smarter about bad-block recovery. The controller maintains a list of bad blocks on the disk.
- The list is initialized during the low-level formatting at the factory and is updated over the life of the disk. Low-level formatting also sets aside spare sectors not visible to the operating system. The controller can be told to replace each bad sector logically with one of the spare sectors. This scheme is known as **sector sparing or forwarding**.
- Swap space is only used as a backing store for pages of memory, which includes memory allocated for the stack, heap, and uninitialized data of a process.
- Solaris now allocates swap space only when a page is forced out of physical memory, rather than when the virtual memory page is first created.
- Linux allows one or more swap areas to be established. A swap area may be in either a swap file on a regular file system or a raw-swap-space partition.

- Each swap area consists of a series of 4-KB page slots which are used to hold swapped pages. Associated with each swap area is a swap map—an array of integer counters.
- If the value of a counter is 0, the corresponding page slot is available. The value of the counter indicates the number of mappings to the swapped page; for example, a value of 3 indicates that the swapped page is mapped to three different processes.

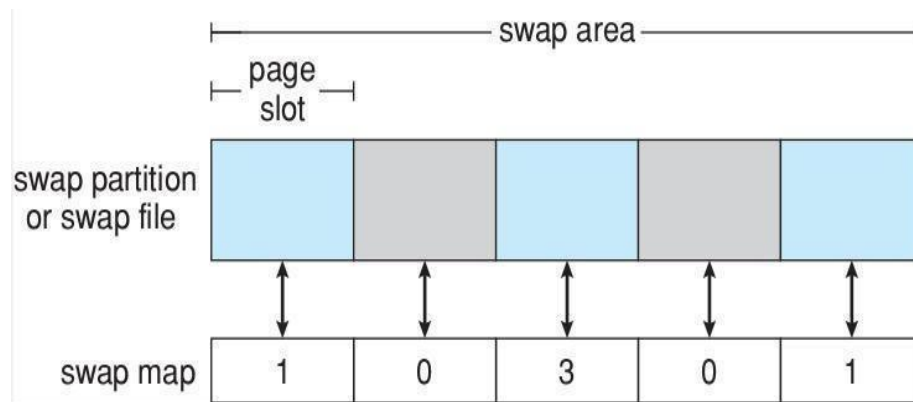


Figure The data structures for swapping on Linux systems.

Goals of Protection

5.17 Protection

- Modern protection concepts have evolved to increase the reliability of any complex system that makes use of shared resources. We need to provide protection for several reasons.
- The most obvious is the need to prevent the mischievous, intentional violation of an access restriction by a user. Of more general importance, however, is the need to ensure that each program component active in a system uses system resources only in ways consistent with stated policies.
- A protection-oriented system provides means to distinguish between authorized and unauthorized usage. The role of protection in a computer system is to provide a mechanism for the enforcement of the policies governing resource use.
- A protection system must have the flexibility to enforce a variety of policies. Policies for resource use may vary by application, and they may change over time.
- For these reasons, protection is no longer the concern solely of the designer of an operating system. The application programmer needs to use protection mechanisms as well, to guard resources created and supported by an application subsystem against misuse.

Principles of Protection

- Frequently, a guiding principle can be used throughout a project, such as the design of an operating system. A key, time-tested guiding principle for protection is the principle of least privilege. It dictates that programs, users, and even systems be given just enough privileges to perform their tasks.
- An operating system following the principle of least privilege implements its features, programs, system calls, and data structures so that failure or compromise of a component does the minimum damage.
- Such an operating system provides mechanisms to enable privileges when they are needed and to disable them when they are not needed.
- Managing users with the principle of least privilege requires creating a separate account for each user, with just the privileges that the user needs. Some systems implement role-based access control (**RBAC**) to provide this functionality.
- Computers implemented in a computing facility under the principle of least privilege can be limited to running specific services, accessing specific remote hosts via specific services, and doing so during specific times.
- Typically, these restrictions are implemented through enabling or disabling each service and through using access control lists.
- The principle of least privilege can help produce a more secure computing environment. Unfortunately, it frequently does not.
- For example, Windows 2000 has a complex protection scheme at its core and yet has many security holes. By comparison, Solaris is considered relatively secure, even though it is a variant of UNIX, which historically was designed with little protection in mind.
- One reason for the difference may be that Windows 2000 has more lines of code and more services than Solaris and thus has more to secure and protect.

5.18 Domain Of Protection

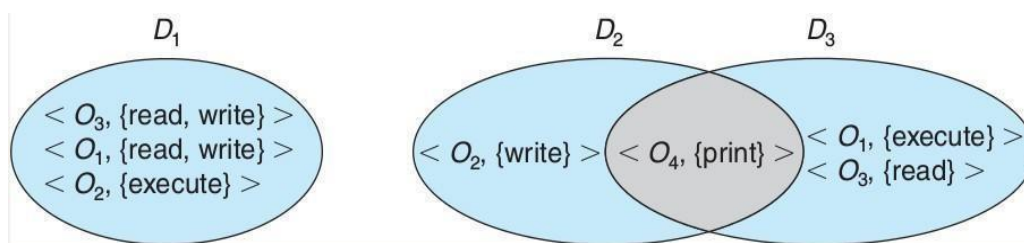
- A computer system is a collection of processes and objects. By objects, we mean both

hardware objects (such as the CPU, memory segments, printers, disks, and tape drives) and software objects (such as files, programs, and semaphores).

- Each object has a unique name that differentiates it from all other objects in the system, and each can be accessed only through well-defined and meaningful operations.
- The operations that are possible may depend on the object. For example, on a CPU, we can only execute. Memory segments can be read and written, whereas a CD-ROM or DVD-ROM can only be read. Data files can be created, opened, read, written, closed, and deleted; program files can be read, written, executed, and deleted.
- A process should be allowed to access only those resources for which it has authorization. Furthermore, at any time, a process should be able to access only those resources that it currently requires to complete its task.
- This second requirement, commonly referred to as the need-to-know principle, is useful in limiting the amount of damage a faulty process can cause in the system.

Domain Structure

- To facilitate the scheme just described, a process operates within a protection domain which specifies the resources that the process may access.
- Each domain defines a set of objects and the types of operations that may be invoked on each object.
- The ability to execute an operation on an object is an access right. A domain is a collection of access rights, each of which is an ordered pair <object-name, rights-set>.
- For example, if domain D has the access right <file F, {read,write}>, then a process executing in domain D can both read and write file F. Domains do not need to be disjoint; they may share access rights.
- For example, in Diagram below, we have three domains: D₁, D₂, and D₃. The access right <O₄, {print}> is shared by D₂ and D₃, implying that a process executing in either of these two domains can print object O₄.



- The association between a process and a domain may be either static, if the set of resources available to the process is fixed throughout the process's lifetime, or dynamic.
- If the association is dynamic, a mechanism must be available to allow domain switching enabling the process to switch from one domain to another. We may also want to allow the content of a domain to be changed.
- A domain can be realized in a variety of ways

Each user may be a domain. In this case, the set of objects that can be accessed depends on the identity of the user. Domain switching occurs when the user is changed -generally when one user logs out and another user logs in.

Each process may be a domain. In this case, the set of objects that can be accessed depends on the identity of the process. Domain switching occurs when one process sends a message to another process and then waits for a response.

Each procedure may be a domain. In this case, the set of objects that can be accessed corresponds to the local variables defined within the procedure. Domain switching occurs

when a procedure call is made.

Domain Implementation (UNIX)

- In the UNIX operating system, a domain is associated with the user. Switching the domain corresponds to changing the user identification temporarily.
- This change is accomplished through the file system as follows. An owner identification and a domain bit (known as the setuid bit) are associated with each file.
- When the domain bit is on, and a user executes that file, the user ID is set to that of the owner of the file; when the bit is off, however, the user ID does not change.
- For example, when a user A (that is, a user with $\text{userID} = A$) starts executing a file owned by B, whose associated domain bit is off, the userID of the process is set to A. When the domain bit is on, the userID is set to that of the owner of the file: B. When the process exits, this temporary userID change ends.
- Even more restrictive, and thus more protective, are systems that simply do not allow a change of user ID. In these instances, special techniques must be used to allow users access to privileged facilities.
- For instance, a daemon process may be started at boot time and run as a special user ID. Users then run a separate program, which sends requests to this process whenever they need to use the facility.

Domain Implementation (MULTICS)

- In the MULTICS system, the protection domains are organized hierarchically into a ring structure. Each ring corresponds to a single domain. The rings are numbered from 0 to 7.
- Let D_i and D_j be any two domain rings. If $j < i$ then D_i is a subset of D_j - That is, a process executing in domain D_j has more privileges than does a process executing in domain D_i . A process executing in domain D_0 has the most privileges.
- A current-ring-number counter is associated with each process, identifying the ring in which the process is executing currently. When a process is executing in ring i , it cannot access a procedure associated with ring j ($j < i$). It can access a procedure associated with ring k ($k \geq i$).

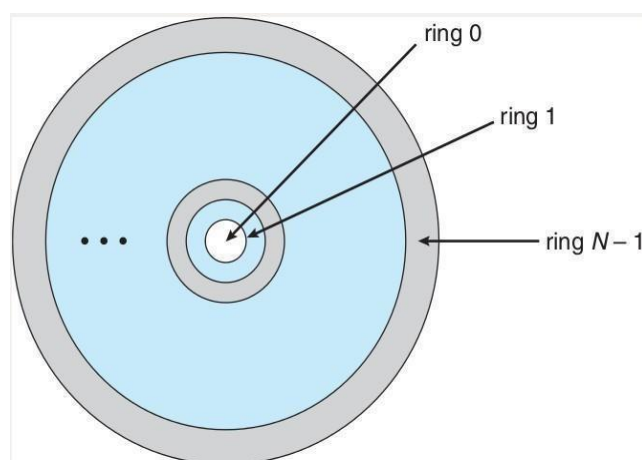


Figure 14.2 MULTICS ring structure.

Multics Benefits and Limits

- Domain switching in MULTICS occurs when a process crosses from one ring to another by calling a procedure in a different ring. Obviously, this switch must be done in a controlled manner; otherwise, a process could start executing in ring 0, and no protection would be provided.
- To allow controlled domain switching, we modify the ring field of the procedure to include the following:

Access bracket. A pair of integers, b_1 and b_2 , such that $b_1 \leq b_2$.

Limit. An integer b_3 such that $b_3 > b_2$.

List of gates. Identifies the entry points (or gates) at which the procedures may be called.

- If a process executing in ring i calls a procedure with access bracket (b_1, b_2) , then the call is allowed if $b_1 \leq i \leq b_2$, and the current ring number of the process remains i . Otherwise, a trap to the operating system occurs, and the situation is handled as follows:
 - If $i < b_1$, then the call is allowed to occur, because we have a transfer to a ring (or domain) with fewer privileges.
 - If $i > b_2$, then the call is allowed to occur only if b_3 is greater than or equal to i and the call has been directed to one of the designated entry points in the list of gates.

5.19 Access Matrix

- Our model of protection can be viewed abstractly as a matrix, called an access matrix. The rows of the access matrix represent domains, and the columns represent objects.
- Each entry in the matrix consists of a set of access rights. The entry $\text{access}(i, j)$ defines the set of operations that a process executing in domain D_i can invoke on object O_j .
- Consider the access matrix shown below. There are four domains and four objects—three files (F_1, F_2, F_3) and one laser printer. A process executing in domain D_1 can read files F_1 and F_3 . A process executing in domain D_4 has the same privileges as one executing in domain D_1 ; but in addition, it can also write onto files F_1 and F_3 . Note that the laser printer can be accessed only by a process executing in domain D_2 .

domain \ object	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

Use of Access Matrix

- The access matrix provides an appropriate mechanism for defining and implementing strict control for both the static and dynamic association between processes and domains.
- When we switch a process from one domain to another, we are executing an operation (switch) on an object (the domain). We can control domain switching by including domains among the objects of the access matrix.
- Similarly, when we change the content of the access matrix, we are performing an operation on an object: the access matrix. Again, we can control these changes by including the access matrix itself as an object.
- Actually, since each entry in the access matrix may be modified individually, we must consider each entry in the access matrix as an object to be protected.
- Processes should be able to switch from one domain to another. Switching from domain D_i to domain D_j is allowed if and only if the access right $\text{switch} \in \text{access}(i, j)$. Thus, according to the access matrix shown below, process executing in domain D_2 can switch to domain D_3 or to domain D_4 . A process in domain D_4 can switch to D_1 , and one in domain D_1 can switch to D_2 .

Access Matrix of above Figure with Domains as Objects

object domain	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

Access Matrix with Copy Rights

object domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute		

(a)

object domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute	read	

(b)

The ability to copy an access right from one domain (or row) of the access matrix to another is denoted by an asterisk (*) appended to the access right.

- The copy right allows the access right to be copied only within the column(that is, for the object) for which the right is defined.
- For example, in Diagram 'a', a process executing in domain D_2 can copy the read operation into any entry associated with file F_2 . Hence, the access matrix of Diagram 'a' can be modified to the access matrix shown in Diagram 'b'. This scheme has two variants:
 - A right is copied from access(i, j) to access(k, j); it is then removed from access(i, j). This action is a transfer of a right, rather than a copy.
 - Propagation of the copy right may be limited. That is, when the right R^* is copied from access(i,j) to access(k,j), only the right R (not R^*) is created. A process executing in domain D_k cannot further copy the right R .

Access Matrix With Owner Rights

object domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		read* owner	read* owner write
D_3	execute		

(a)

object domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		owner read* write*	read* owner write
D_3		write	write

(b)

We also need a mechanism to allow addition of new rights and removal of some rights. The owner right controls these operations.

- If access(i, j) includes the owner right, then a process executing in domain D_i can add and remove any right in any entry in column j .
- For example, in Diagram 'a', domain D_1 is the owner of F_1 and thus can add and delete any valid right in column F_1 . Similarly, domain D_2 is the owner of F_2 and F_3 and thus can add and remove
- any valid right within these two columns.
- Thus, the access matrix of Diagram 'a' can be modified to the access matrix shown in Diagram 'b'.
- The copy and owner rights allow a process to change the entries in a column. A mechanism is also needed to change the entries in a row.
- The control right is applicable only to domain objects. If access(i, j) includes the control right, then a process executing in domain D_i can remove any access right from row j .
- For example, suppose that, we include the control right in access(D_2, D_4). Then, a process executing in domain D_2 could modify domain D_4 , as shown below.

Modified Access Matrix

object domain	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

object domain	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch control
D_3		read	execute					
D_4	write		write		switch			

*******UNIVERSITY QP QUESTIONS: *******

1. Given the following queue 95, 180, 34, 119, 11, 123, 62, 64 with head initially at track 50 and ending at track 199 calculate the number of moves using FCFS, SSTF, Elevator and C look.
2. Let a disk drive has 5000 cylinders from 0 to 4999. Currently drive is at 143rd cylinder, and the previous request was at cylinder 125. Queue of pending request in FIFO order is 86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130. What is the total distance the disk arm moves to satisfy all the pending requests for each of the following disk scheduling algorithms from current position i)FCFS ii) SCAN iii) LOOK
3. Suppose that a disk has 50 cylinders named 0 to 49. The R/W head is currently serving at cylinder 15. The queue of pending requests are in order :4, 40, 11, 35, 7, 14 starting from the current head position, what is the total distance traveled (in cylinders) by the disk arm to satisfy the requests using algorithms FCFS, SSTF and LOOK. Illustrate with figure in each case.
4. Explain the various disk Schedule algorithms with examples.
5. Explain the following disk scheduling algorithm in brief i)SSTF ii) SCAN iii) LOOK
6. Explain in brief, the selection of a disk scheduling algorithm
7. What is disk scheduling? Discuss different disk scheduling techniques
8. Explain the capability lists methods of implementing access matrix
9. What are access matrices? Explain its implementation
10. Differentiate between security and protection
11. Discuss the strength and weakness of implementing an access matrix using access lists that are associated with objects
12. Explain the difference between protection and security? Describe the scheme of capability lists to implement protection
13. Explain Access Matrix method of system protection
14. Write a note on: i) Domain of protection ii) Access matrix